

## LAB 5 — Mini-MapReduce on Amazon Elastic MapReduce (EMR)

**Estimated time:** 3 hours

**Work mode:** Individual

**Submission:** Defense with report + code repository

**Platform:** Amazon EMR (managed Hadoop)

---

### 1. Lab Objectives

By completing this lab, students will be able to:

- Explain the **MapReduce programming model**.
- Run **MapReduce jobs** on a managed cluster (Amazon EMR).
- Understand **data distribution, parallelism, and fault tolerance**.
- Use **HDFS as distributed storage**.
- Compare **local execution vs distributed execution**.
- Analyze **job performance and scalability**.

This lab supports CLOs related to:

- big-data processing,
  - cloud deployment,
  - performance and scalability analysis,
  - integration of Distributed Systems concepts with data frameworks.
- 

### 2. What You Will Build

You will run a **Mini-MapReduce pipeline** on EMR that:

- Processes a **large text dataset** stored in HDFS.
  - Executes a **MapReduce job** (e.g., WordCount).
  - Produces aggregated output back to HDFS.
  - Demonstrates **parallel (distributed) execution across multiple nodes**.
- 

### 3. Technology Stack

- **Amazon EMR**
- **Hadoop MapReduce**
- **HDFS**
- **SSH / AWS Console**
- **Python (preferred)** or Java

---

## 4. Dataset

You should use the following:

- Wikipedia text dump (small sample) - <https://github.com/LGDoor/Dump-of-Simple-English-Wiki> (read the description)

Upload dataset to local file system and copy it to HDFS (SSH into master node first):

```
wget https://github.com/LGDoor/Dump-of-Simple-English-Wiki/raw/refs/heads/master/corpus.tgz
```

Verify the file and extract the archive:

```
ls -lh corpus.tgz
tar -xvzf corpus.tgz
```

Create a directory in HDFS and copy the extracted files to HDFS:

```
hdfs dfs -mkdir -p /user/hadoop/input
hdfs dfs -put corpus.txt /user/hadoop/input/
```

Verify Files in HDFS:

```
hdfs dfs -ls /user/hadoop/input/
```

---

## 5. Cluster Setup (Amazon EMR)

### Step 1 — Create EMR (Launch AWS Academy Learner Lab)

- Start Lab and find EMR service in AWS Management Console.
- This service can assume the LabRole IAM role.
- Supported Instance types: nano, micro, small, medium, and large. If you attempt to launch a larger instance type, it will be terminated.
- **Tip:** If you have any trouble successfully launching a cluster, try using the m4.large instance type. *Example* EMR cluster configuration details:
  - *Cluster configuration:* Instance groups, Primary: **m4.large**, Core: **m4.large**, Task -1: **m4.large**
  - *Provisioning configuration:* Core: 1 instance and Task -1: 1 instance
  - *Security configuration:* EC2 key pair: **vockey**
  - *IAM roles:* Amazon EMR service role: choose existing: **EMR\_DefaultRole**, EC2 instance profile for Amazon EMR: choose existing: **EMR\_EC2\_DefaultRole**
- **Note:** If you encounter the issue with S3 bucket creation as part of cluster creation:
  - Expand the section - **Cluster logs** and **Uncheck** the value for *Publish cluster-specific logs to Amazon S3*, and proceed with cluster creation. The logs will not be stored in s3 bucket, but cluster will get created.

- Maximum of 32 vCPU used by concurrently running EC2 instances in an AWS Region. Note that you are also limited to launching no more than nine (9) instances (of any size) in a Region at once.
  - **Note:** *An EMR cluster will not continue to work if your session ends.* In Learner Labs, session end causes the EC2 instances that the EMR cluster uses to be stopped, and stopping an EMR cluster is not supported (by AWS). *Recommendation:* write EMR job results to S3 if you need to preserve your results, before you end your current Learner Labs session, then read the results back into a new EMR cluster as needed when you start your next Learner Labs session.
  - Applications:
    - Hadoop
    - Hive (optional)
- 

## Step 2 — Verify Cluster

SSH into master node:

```
ssh -i key.pem hadoop@<master-public-dns>
```

Check cluster:

```
yarn node -list
```

```
hdfs dfsadmin -report
```

Create HDFS output directory:

```
hdfs dfs -mkdir -p /user/hadoop/output/
```

---

## 6. Part 1 — MapReduce Job

1. **Word Count Streaming using MapReduce + Python** (follow the instructions in [this](#) page)
- 

### Programming Model

- **Map:** process input splits, emit key–value pairs
  - **Shuffle:** handled by framework
  - **Reduce:** aggregate values by key
- 

## 7. Run Job on EMR

Example Hadoop streaming job:

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \  
-input /user/hadoop/input/ \  
-output /user/hadoop/output/ \  

```

```
-mapper mapper.py \  
-reducer reducer.py \  
-files mapper.py,reducer.py
```

---

## 8. Part 2 — Experimentation

You must perform **at least one experiment**:

### Scenario A — Scaling

- Run job with 2 core nodes
- Run job with 4 core nodes
- Compare runtime

### Scenario B — Input Size

- Small dataset vs large dataset
- Observe job duration

### Scenario C — Failure Tolerance

- Terminate one worker node
- Observe job recovery behavior

Explain observations **verbally in the demo**.

---

## 9. Output Validation

Check output:

```
hdfs dfs -ls -p /user/hadoop/output/  
hdfs dfs -head /user/hadoop/output/*
```

Verify correctness.

---

## 10. Deliverables

### Deliverable 1 — Defense with report

You must show:

1. EMR cluster configuration
2. Job submission
3. Job execution (YARN UI optional)
4. Output in HDFS
5. One experiment (Section 8)
6. Verbal explanation (no slides)

---

## Deliverable 2 — Code Repository

Repository must include:

- Mapper & reducer code
- README with run command
- Dataset description

---

### 11. Common Mistakes to Avoid

- Running job locally instead of EMR
- Forgetting to use HDFS for input/output
- Using very small datasets
- Not showing parallelism / distributed computing